

<pre>import re import numpy as np import pandas as pd import matplotlib.pyplot as plt import seaborn as sns from scipy import integrate, signal from scipy.stats import linregress sns.set_theme(style="whitegrid", palette="deep") plt.rcParams.update({"figure.dpi": 130, "figure.figsize": (11, 6.5)})</pre>	This imports all necessary function sfor use. Lines are assigned to a graph and the color pallettee is decided.
<pre>print("\n" + "=" * 70) print(" MODULE A — Aggregate Gradation Automator") print("=" * 70) raw = pd.read_excel(filepath, sheet_name=0, header=None)</pre>	This section first prints a formatted title banner to separate Module A from the rest of the program output. The repeated "=" * 70 creates horizontal divider lines, while the middle print statement displays the module name “Aggregate Gradation Automator.” After the title is displayed, the code reads the Excel spreadsheet into a pandas dataframe named raw using pd.read_excel().
<pre>header_row = raw.iloc[1] sieve_sizes_mm = np.array([float(header_row.iloc[c]) for c in range(2, raw.shape[1]) if pd.notna(header_row.iloc[c])])</pre>	raw.iloc[1] selects the second row of the data frame and stores it as header row. The list comprehension then loops through each column starting at column index 2 and checks whether the cell contains valid data
<pre>agg_names, blend_pcts, agg_passing = [], [], [] spec_min_vals, spec_max_vals = None, None spec_count = 0</pre>	This section creates variables that will be used to store aggregate and specification data throughout the program. A counter is created for tracking.

<pre> for _, row in raw.iloc[2:].iterrows(): name_raw = str(row.iloc[0]).strip() pct_raw = row.iloc[1] # Specification rows: name is NaN if name_raw in ("nan", ""): spec_vals = np.array([float(row.iloc[c]) if pd.notna(row.iloc[c]) else np.nan for c in range(2, 2 + len(sieve_sizes_mm))]) if spec_count == 0: spec_min_vals = spec_vals elif spec_count == 1: spec_max_vals = spec_vals spec_count += 1 continue </pre>	This loops the data row by row, it checks the conditions and stores the data for later use. If i is found, then the row is skipped and calculations are stopped.
<pre> pct_str = str(pct_raw).strip() if pct_str.lower() in ("nan", "?", "sapecification range", ""): continue # Skip the combined-gradation row if "combined" in name_raw.lower(): continue try: pct = float(pct_str) except ValueError: continue passing = np.array([float(row.iloc[c]) if pd.notna(row.iloc[c]) else np.nan for c in range(2, 2 + len(sieve_sizes_mm))]) agg_names.append(name_raw) blend_pcts.append(pct) agg_passing.append(passing) </pre>	Checks for missing or invalid entries, skips rows that should not be processed, and converts the remaining data into numerical values. It then extracts the passing values for each sieve size and saves the aggregate name, blend percentage, and gradation data
<pre> def _check(row): c = row["Combined_%Pass"] lo = row["Spec_Min_%"] hi = row["Spec_Max_%"] if pd.isna(lo) or pd.isna(hi): return "—" return "PASS" if lo <= c <= hi else "FAIL" </pre>	Compare the calculated percent passing value to the minimum and maximum specification ranges. If specification values are missing, the function returns a

	placeholder symbol. Otherwise, it determines whether the value passes or fails,
<pre> for i, (name, pct) in enumerate(zip(agg_names, blend_pcts)): ax.plot(sieve_sizes_mm, agg_passing[i] * 100, linestyle="--", linewidth=1.2, color=palette[i], label=f"{name} ({pct * 100:.0f}%)", alpha=0.7) ax.plot(sieve_sizes_mm, combined * 100, color="black", linewidth=2.5, marker="o", markersize=5, label="Combined Gradation", zorder=5) </pre>	Plots each individual aggregate gradation curve, then plots the combined gradation curve on top so it can be compared to the specification limits. After curves are plotted the graph details are assigned.
<pre> if spec_min_vals is not None and spec_max_vals is not None: valid = ~np.isnan(spec_min_vals) & ~np.isnan(spec_max_vals) if valid.any(): ax.fill_between(sieve_sizes_mm[valid], spec_min_vals[valid] * 100, spec_max_vals[valid] * 100, alpha=0.15, color="green", label="NDOT Spec Band") ax.plot(sieve_sizes_mm[valid], spec_min_vals[valid] * 100, color="green", linewidth=1.5, linestyle=":", label="Spec Min") ax.plot(sieve_sizes_mm[valid], spec_max_vals[valid] * 100, color="green", linewidth=1.5, linestyle=":", label="Spec Max") ax.set_xscale("log") pos_sieve = sieve_sizes_mm[sieve_sizes_mm > 0] ax.set_xlim(sieve_sizes_mm.max() * 1.3, pos_sieve.min() * 0.4) ax.set_xlabel("Sieve Size (mm) — Log Scale", fontsize=12) ax.set_ylabel("% Passing", fontsize=12) ax.set_title("Module A — Aggregate Gradation Curve\ncvs. NDOT Control Points", fontsize=13) ax.legend(loc="upper left", fontsize=8, ncol=2) ax.set_ylim(-1, 106) ax.grid(True, which="both", linestyle=":", alpha=0.5) plt.tight_layout() plt.savefig("Module_A_Gradation.png", bbox_inches="tight") plt.show() print("\n Plot saved: Module_A_Gradation.png") return result_df </pre>	
<pre> sample_header_row = raw.iloc[0].tolist() sample_groups = [] for col_idx, cell in enumerate(sample_header_row): if pd.isna(cell) and "Sample" in str(cell): </pre>	Scans through the header cells looking for sample labels, then extracts important information

<pre> t_m = re.search(r"\t\s*=\s*([\d.]+)", str(cell)) d_m = re.search(r"D\s*=\s*([\d.]+)", str(cell)) sample_groups.append({ "name": f"Sample {len(sample_groups) + 1}", "col_load": col_idx, "col_disp": col_idx + 1, "thickness": float(t_m.group(1)) if t_m else 61.91, "diameter": float(d_m.group(1)) if d_m else 150.0, </pre>	such as sample thickness, diameter, and the column locations of the load. The data is stored for later use.
<pre> FILE_AGG = "Aggregate_Gradation_csv.xlsx" FILE_VOL = "Volumetrics_csv.xlsx" FILE_IDEAL = "IDEAL-CT_csv.xlsx" FILE_HWTT = "HWTT_csv.xlsx" df_a = run_module_a(FILE_AGG) df_b = run_module_b(FILE_VOL) df_c = run_module_c(FILE_IDEAL) df_d = run_module_d(FILE_HWTT) print("\n" + "=" * 70) print(" All four modules complete.") print(" Output plots: Module_A_Gradation.png") print(" Module_B_Volumetrics.png") print(" Module_C_IDEAL_CT.png") print(" Module_D_HWTT.png") print("=" * 70) </pre>	First defines the filenames for the input datasets used in each module. It then runs all four analysis modules one after another, storing the results from each module into separate dataframes. Once the program is completed a summary page is provided.
<pre> data_rows = raw.iloc[2:].reset_index(drop=True) results_list = [] fig, ax = plt.subplots(figsize=(12, 7)) colors_ct = sns.color_palette("deep", n_colors=len(sample_groups)) for k, sg in enumerate(sample_groups): raw_load = pd.to_numeric(data_rows.iloc[:, sg["col_load"]], errors="coerce") raw_disp = pd.to_numeric(data_rows.iloc[:, sg["col_disp"]], errors="coerce") mask = raw_load.notna() & raw_disp.notna() load = raw_load[mask].to_numpy(dtype=float) disp = raw_disp[mask].to_numpy(dtype=float) </pre>	The data is grouped by county to analyze by county in Ohio. The summary statistics are then calculated. The average tornado and landslide risk is provided for each county. The data is stored in oh_grouped_data and then displayed for verification.
<pre> FILE_AGG = "Aggregate_Gradation_csv.xlsx" FILE_VOL = "Volumetrics_csv.xlsx" FILE_IDEAL = "IDEAL-CT_csv.xlsx" FILE_HWTT = "HWTT_csv.xlsx" df_a = run_module_a(FILE_AGG) df_b = run_module_b(FILE_VOL) df_c = run_module_c(FILE_IDEAL) df_d = run_module_d(FILE_HWTT) print("\n" + "=" * 70) </pre>	The data is grouped by county to analyze by county in Missouri. The summary statistics are then calculated. The average tornado and landslide risk is provided for each county. The data is stored in

<pre>print(" All four modules complete.") print(" Output plots: Module_A_Gradation.png") print(" Module_B_Volumetrics.png") print(" Module_C_IDEAL_CT.png") print(" Module_D_HWTT.png") print("=" * 70)</pre>	mo_grouped_data and then displayed for verification.
<pre>oh_grouped_data.plot(kind='bar', figsize=(20, 10), width=0.8) # Customizing the plot plt.xlabel('County') plt.ylabel('Mean Risk Value') plt.title('Ohio: Comparison of Risk of Tornadoes vs. Landslides') plt.xticks(rotation=90) # Show the plot plt.legend(title="Risk Type") # Adding a legend to distinguish risk types plt.show()</pre>	A bar graph is created showing mean landslide and tornado risk for each county in Ohio. The plot is then given an x and y label, a title, and rotates county names for readability. A legend is created to distinguish between the types of risk, and the plot is presented.
<pre>mo_grouped_data.plot(kind='bar', figsize=(20, 10), width=0.8) # Customizing the plot plt.xlabel('County') plt.ylabel('Mean Risk Value') plt.title('Missouri: Comparison of Risk of Tornadoes vs. Landslides') plt.xticks(rotation=90) # Show the plot plt.legend(title="Risk Type") # Adding a legend to distinguish risk types plt.show()</pre>	A bar graph is created showing mean landslide and tornado risk for each county in Missouri. The plot is then given an x and y label, a title, and rotates county names for readability. A legend is created to distinguish between the types of risk, and the plot is presented.
<pre>oh_complete_data['tornado_risk'] = oh_complete_data['TRND_EALT']*oh_complete_data['SOVI_SCORE'] / oh_complete_data['RESL_SCORE'] #print(oh_complete_data) print(oh_complete_data['tornado_risk'])</pre>	This creates a new variable called tornado_risk that combines hazard, vulnerability, and resilience factors. Expected loss is multiplied by vulnerability and divided by resilience. The data is stored as a new column. The tornado risk values for Ohio are then displayed.
<pre>#plot the data plt.figure plt.boxplot(oh_complete_data['tornado_risk']) plt.title("Ohio Tornado Risk Distribution") plt.ylabel("Risk Index") plt.show()</pre>	A blank figure is created for use. A boxplot is created to plot the tornado risk for Ohio. A title is created and the

	graph is given a y label. The plot is then presented.
<pre>oh_complete_data['landslide_risk'] = oh_complete_data['LNDS_EALT']*oh_complete_data['SOVI_SCORE'] / oh_complete_data['RESL_SCORE'] #print(oh_complete_data) print(oh_complete_data['landslide_risk'])</pre>	This creates a new variable called landslide_risk that combines hazard, vulnerability, and resilience factors. Expected loss is multiplied by vulnerability and divided by resilience. The data is stored as a new column. The landslide risk values for Ohio are then displayed.
<pre>#plot the data plt.figure plt.boxplot(oh_complete_data['landslide_risk']) plt.title("Ohio Landslide Risk Distribution") plt.ylabel("Risk Index") plt.show()</pre>	A blank figure is created for use. A boxplot is created to plot the tornado risk for Ohio. A title is created and the graph is given a y label. The plot is then presented.
<pre>mo_complete_data['tornado_risk'] = mo_complete_data['TRND_EALT']*mo_complete_data['SOVI_SCORE'] / mo_complete_data['RESL_SCORE'] #print(mo_complete_data) print(mo_complete_data['tornado_risk'])</pre>	This creates a new variable called tornado_risk that combines hazard, vulnerability, and resilience factors. Expected loss is multiplied by vulnerability and divided by resilience. The data is stored as a new column. The tornado risk values for Missouri are then displayed.
<pre>#plot the data plt.figure plt.boxplot(mo_complete_data['tornado_risk']) plt.title("Missouri Tornado Risk Distribution") plt.ylabel("Risk Index") plt.show()</pre>	A blank figure is created for use. A boxplot is created to plot the tornado risk for Ohio. A title is created and the graph is given a y label. The plot is then presented.

<pre>mo_complete_data['landslide_risk'] = mo_complete_data['LNDS_EALT']*mo_complete_data['SOVI_SCORE'] / mo_complete_data['RESL_SCORE'] #print(mo_complete_data) print(mo_complete_data['landslide_risk'])</pre>	This creates a new variable called landslide_risk that combines hazard, vulnerability, and resilience factors. Expected loss is multiplied by vulnerability and divided by resilience. The data is stored as a new column. The landslide risk values for Missouri are then displayed.
<pre>plt.figure plt.boxplot(mo_complete_data['landslide_risk']) plt.title("Missouri Landslide Risk Distribution") plt.ylabel("Risk Index") plt.show()</pre>	A blank figure is created for use. A boxplot is created to plot the tornado risk for Ohio. A title is created and the graph is given a y label. The plot is then presented.
<pre>## Read the Census Tracts data tracts = gpd.read_file('NRI_Shapefile_CensusTracts.shp') ##Verifications for Shape files tracts.head() ###This checks for Coordinate Reference System (CRS) tracts.crs ###Checks the geometries in the data tracts.geometry.head()</pre>	The Shapefile is read and stored in a GeoDataFrame called tracts. The first few rows of the data are displayed. The Coordinates reference system is then checked, which defines the spatial data's mapping. The geometry data is then displayed.
<pre>tracts = gpd.read_file('NRI_Shapefile_CensusTracts.shp') state_map = tracts[tracts["STATE"] == "Ohio"].copy() state_map = state_map.replace(-9999, np.nan) state_map = state_map.to_crs("ESRI:102003") fig, ax = plt.subplots(1, 1, figsize=(15, 10)) state_map.plot(column="TRND_RISKS",</pre>	Loads census tract Shapefile into a GeoDataFrame. Filters data to include only Ohio. Missing data is replaced with NaN as to not impact the map. The coordinates are then converted to CRS, which improves map display. A figure is created for plotting. The map for the plot

<pre> cmap="RdYlBu_r", legend=True, ax=ax, legend_kwds={"label": "Risk Score (0-100)", "orientation": "horizontal", "shrink":0.6,"pad":0.05}, missing_kwds={"color": "lightgrey"}) ax.set_title("Ohio Tornado Risk Index") ax.set_axis_off() plt.show() </pre>	is then generated. The column tornado risks is then used for data. Each tract is then assigned as a color based on tornado risk. It is given a red and blue gradient. A legend is then added and customized. It is then given a title and axis are turned off, and the data is displayed.
<pre> tracts = gpd.read_file('NRI_Shapefile_CensusTracts.shp') state_map = tracts[tracts["STATE"] == "Ohio"].copy() state_map = state_map.replace(-9999, np.nan) state_map = state_map.to_crs("ESRI:102003") fig, ax = plt.subplots(1, 1, figsize=(15, 10)) state_map.plot(column="LND_S_RISKS", cmap="RdYlBu_r", legend=True, ax=ax, legend_kwds={"label": "Risk Score (0-100)", "orientation": "horizontal", "shrink":0.6,"pad":0.05}, missing_kwds={"color": "lightgrey"}) ax.set_title("Ohio Landslide Risk Index") ax.set_axis_off() plt.show() </pre>	Loads census tract Shapefile into a GeoDataFrame. Filters data to include only Ohio. Missing data is replaced with NaN as to not impact the map. The coordinates are then converted to CRS, which improves map display. A figure is created for plotting. The map for the plot is then generated. The column landslide risks is then used for data. Each tract is then assigned as a color based on landslide risk. It is given a red and blue gradient. A legend is then added and customized. It is then given a title and axis are turned off, and the data is displayed.
<pre> tracts = gpd.read_file('NRI_Shapefile_CensusTracts.shp') state_map = tracts[tracts["STATE"] == "Missouri"].copy() state_map = state_map.replace(-9999, np.nan) state_map = state_map.to_crs("ESRI:102003") fig, ax = plt.subplots(1, 1, figsize=(15, 10)) state_map.plot(</pre>	Loads census tract Shapefile into a GeoDataFrame. Filters data to include only Missouri. Missing data is replaced with NaN as to not impact the map. The coordinates are then converted to CRS, which improves map display. A figure is created for plotting. The map for the plot

<pre>column="TRND_RISKS", cmap="RdYlBu_r", legend=True, ax=ax, legend_kwds={"label": "Risk Score (0-100)", "orientation": "horizontal", "shrink":0.6,"pad":0.05}, missing_kwds={"color": "lightgrey"}) ax.set_title("Missouri Tornado Risk Index") ax.set_axis_off() plt.show()</pre>	is then generated. The column tornado risks is then used for data. Each tract is then assigned as a color based on tornado risk. It is given a red and blue gradient. A legend is then added and customized. It is then given a title and axis are turned off, and the data is displayed.
<pre>tracts = gpd.read_file("NRI_Shapefile_CensusTracts.shp") state_map = tracts[tracts["STATE"] == "Missouri"].copy() state_map = state_map.replace(-9999, np.nan) state_map = state_map.to_crs("ESRI:102003") fig, ax = plt.subplots(1, 1, figsize=(15, 10)) state_map.plot(column="LNDS_RISKS", cmap="RdYlBu_r", legend=True, ax=ax, legend_kwds={"label": "Risk Score (0-100)", "orientation": "horizontal", "shrink":0.6,"pad":0.05}, missing_kwds={"color": "lightgrey"}) ax.set_title("Missouri Landslide Risk Index") ax.set_axis_off() plt.show()</pre>	Loads census tract Shapefile into a GeoDataFrame. Filters data to include only Missouri. Missing data is replaced with NaN as to not impact the map. The coordinates are then converted to CRS, which improves map display. A figure is created for plotting. The map for the plot is then generated. The column landslide risks is then used for data. Each tract is then assigned as a color based on landslide risk. It is given a red and blue gradient. A legend is then added and customized. It is then given a title and axis are turned off, and the data is displayed.